EXECUTIVE OPERATIONAL BLUEPRINT

HERMES PLATFORM :: MASTER DOSSIER

CONSOLIDATED ENGINEERING SERIES · UNABRIDGED · DOCUMENTS 1-5

► **DOSSIER CONTENTS (UNABRIDGED)**

- DOC 1: HPR-01 // TECHNICAL ROADMAP
- DOC 2: RFC-001 // DETERMINISTIC KERNEL
- DOC 3: RFC-002 // CANONICAL EVENT MODEL
- DOC 4/5: RFC-003/004 // ALGEBRAIC ORDERING & DISTRIBUTED CLOSURE

STATUS: CANONICAL. ALL INVARIANTS, SCHEMAS, AND TOPOLOGICAL BOUNDARIES ENFORCED.

ENGINEERING PLANNING DOCUMENT

DOCUMENT 1 :: TECHNICAL ROADMAP

HERMES PLATFORM ENGINEERING SERIES · REF: HPR-01

DOCUMENT ID	HPR-01	**EFFECTIVE DATE**	June 2026
REVISION	v1.0	**STATUS**	Draft
CLASSIFICATION	Internal Engineering Ref	**DEPARTMENT**	Core Engineering

01 EXECUTIVE SUMMARY

Hermes is no longer a “workflow engine” project. It is evolving into a deterministic runtime platform, with agent execution as merely one application layer built on top of a proven kernel. This document defines the phased engineering roadmap from kernel validation through to distributed multi-node federation.

The roadmap comprises seven phases (Phase 0 through Phase 6), each with explicit objectives, deliverables, execution flows, and success criteria. The strategic foundation of the entire platform rests on Phase 0: proving that Hermes can deterministically reconstruct reality from facts. Everything else is built on top of that kernel.

> ⚠ **STRATEGIC PRIORITY**
> No subsequent phase may begin until Phase 0 success criteria are fully demonstrated. The deterministic kernel is the non-negotiable foundation of the entire platform.

02 STRATEGIC CONTEXT

The core architectural thesis of Hermes is this: most agent frameworks begin by building agents, then attempt to retrofit governance, observability, and determinism. Hermes inverts this. The deterministic kernel is built first. Governance is built second. Agents are introduced only after the substrate beneath them is proven correct and mature.

The strategic inflection point is between Phase 0 and Phase 1. Once the event stream → deterministic replay → identical state reconstruction chain is validated, the hardest and most differentiated part of the platform is complete. All subsequent phases add capabilities around a proven kernel, rather than discovering the kernel while simultaneously building agents, UIs, orchestration, and governance.

03 ROADMAP OVERVIEW

PHASE	NAME	CORE QUESTION	STATUS
0	Prove the Kernel	Can Hermes deterministically reconstruct reality from facts?	● Active
1	Constitutional Runtime	Can Hermes enforce governance before execution?	Planned
2	Execution Substrate	Can Hermes execute real work?	Planned
3	Compilation	Can intent compile into execution?	Planned
4	Agent Integration	Can probabilistic systems safely participate?	Planned
5	Council & Governance	Can the system evolve without compromising determinism?	Planned
6	Distributed Hermes	Can Hermes operate across many nodes?	Future

04 PHASE DEFINITIONS

PHASE 0: PROVE THE KERNEL

• ACTIVE

OBJECTIVE

Can Hermes deterministically reconstruct reality from facts?

RATIONALE

This is the foundational proof-of-concept. If this phase fails or is bypassed, everything built above it rests on sand. No other phase begins until Phase 0 success criteria are met.

DELIVERABLES

- Event model & Event store
- Reducer & Replay engine
- Checkpoints & Policy evaluator

SUCCESS CRITERIA

- Run a full execution trace
- Persist all events to the event store
- Destroy all in-memory state
- Replay the trace from persisted events
- Reconstruct an identical state to the original

PHASE 1: ESTABLISH THE CONSTITUTIONAL RUNTIME

PLANNED

OBJECTIVE

Can Hermes enforce governance before execution?

EXECUTION FLOW

`Decision → Constitution → Policy → Execution`

DELIVERABLES

ConstitutionIR, PolicyIR, Evaluation engine, Approval workflows, Constraint application, Audit chain validation.

SUCCESS CRITERIA

Denied actions never execute; Approval gates block execution correctly; Constrained execution is fully reproducible; Replay produces identical verdicts.

PHASE 2: BUILD THE EXECUTION SUBSTRATE

PLANNED

OBJECTIVE

Can Hermes execute real work?

EXECUTION FLOW

`DecisionEvent → Policy Boundary → ExecutionEvent → SystemEvent`

DELIVERABLES

Tool registry, Tool contracts, Execution sandbox, Retry engine, Failure recovery, Resource accounting.

SUCCESS CRITERIA

Retries are deterministic/reproducible; Failures are fully observable; Execution provenance is complete end-to-end.

PHASE 3: INTRODUCE COMPILATION

PLANNED

OBJECTIVE

Can intent compile into execution?

EXECUTION FLOW

```
Intent → HermesIR → Compiler → ExecIR
→ PlanSnapshot
```

DELIVERABLES

HermesIR schema, ExecIR schema, Compiler pipeline, Snapshot generation, Dependency pinning.

SUCCESS CRITERIA

Identical IR input produces identical execution plans; Plans are replay-safe across runtime restarts; Snapshots remain valid across upgrades.

PHASE 4: INTRODUCE AGENTS

PLANNED

OBJECTIVE

Can probabilistic systems safely participate?

EXECUTION FLOW

```
Model → Proposal → DecisionEvent →
Policy → Execution
```

DELIVERABLES

Agent abstraction layer, Model adapters, Candidate generation, Decision normalization, Multi-agent coordination.

> ■ **CRITICAL CONSTRAINT — AGENT BOUNDARIES**
>
> Agents never mutate state. Agents never execute tools. Agents only propose. This constraint is non-negotiable. Agents are probabilistic; the kernel must remain deterministic.

PHASE 5: COUNCIL AND GOVERNANCE

PLANNED

OBJECTIVE

Can the system evolve without compromising determinism?

EXECUTION FLOW

```
Observation → Proposal → Review →
Compilation → New Snapshot
```

DELIVERABLES

Council architecture, GovernanceProposal system, Patch review workflows, Constitutional amendment process, Signed proposals.

PHASE 6: DISTRIBUTED HERMES

FUTURE

OBJECTIVE

Can Hermes operate across many nodes?

OPEN DESIGN QUESTIONS

- Single writer model vs. multi-writer?
- Leader election strategy?
- Consensus algorithm selection?
- CRDT applicability?

DELIVERABLES

Distributed event log, Vector clocks, Trace partitioning, Consensus model, Multi-runtime federation.

05 LONG-TERM ARCHITECTURE: MATURE HERMES STACK

The following diagram represents the mature Hermes stack in its target state, with the highest-level application layer at the top and the foundational event infrastructure at the bottom. Each layer is strictly dependent on the correctness of the layer below it.

LAYER (TOP → BOTTOM)	PHASE INTRODUCED	DESCRIPTION
Applications	Post Phase 6	End-user and system-facing application logic built on the full platform
Agent Systems	Phase 4	Probabilistic model layer; constrained to proposal generation only
Council Governance	Phase 5	Controlled system evolution via signed, auditable governance proposals
HermesIR / ExecIR Compiler	Phase 3	Compilation pipeline from high-level intent to deterministic execution plans
Policy & Constitution Layer	Phase 1	Constitutional and policy evaluation; governance enforcement before execution
Deterministic Kernel	Phase 0	Core runtime; state reconstruction via event application (reducer model)
Event Store + Replay Core	Phase 0	Append-only persisted event log; foundation of all determinism guarantees

06 PHASE GATE POLICY

Each phase has explicit success criteria that must be fully demonstrated before the subsequent phase begins. The following rules govern phase transitions:

- No phase may be initiated in parallel with an incomplete prior phase gate, with the exception of preparatory design work.
- Engineering velocity must not be used as justification for bypassing phase gates.
- Phase gate demonstrations must be reproducible and reviewed by the senior engineering team before sign-off.
- Any phase gate failure requires root-cause analysis before the phase may be reattempted.
- Phase 6 remains unscheduled until all Phase 0–5 gates have been closed.

> ⚠ **WARNING**
>
> Bypassing a phase gate for any reason — including external pressure, milestone deadlines, or perceived low risk — is a platform integrity violation. The entire value of the Hermes architecture depends on the correctness of each layer before the next is built upon it.

IMPLEMENTATION SPECIFICATION

DOCUMENT 2 :: RFC-001 DETERMINISTIC KERNEL, EXECUTABLE CONSTITUTIONS, CAUSAL CLOSURE

HERMES PLATFORM ENGINEERING SERIES · REF: RFC-001

DOCUMENT ID	RFC-001	**EFFECTIVE DATE**	June 14, 2026
REVISION	v1.4	**STATUS**	Canonical Draft
CLASSIFICATION	Internal Engineering Ref	**SCOPE**	Core Execution Model

1. ABSTRACT

This RFC defines the Hermes Runtime as a deterministic, replayable state machine governed by a strict hierarchy of compiled intermediate representations, anchored by an executable constitution.

The runtime treats LLMs as proposal generators. All actions are filtered through synchronous policy evaluation, executed as side-effects, and recorded as causal facts. The kernel reducer derives reality exclusively from state-transition facts (SystemEvent), which must provably trace their origin to execution attempts (ExecutionEvent).

This RFC formalizes:

- An executable ConstitutionIR with strict precedence and severity.
- Causal closure between execution attempts and state transitions.
- A primitive, explicitly typed state-transition taxonomy.
- An event-sourced blackboard projection model.
- Cryptographic authorship via signatures.
- The TraceManifest as the genesis block of execution.

2. GOALS

The Hermes Runtime exists to support workflows that are:

- Deterministic under replay with bounded recovery
- Causally closed (no orphaned state mutations)
- Constitutionally supreme and policy-governed
- Auditable with full provenance and cryptographic authorship
- Robust to probabilistic model output, retries, and partial failure

3. TERMINOLOGY

TERM	DEFINITION
ConstitutionIR	The runtime-wide axiomatic law. Compiled into executable validators. Overrides all other IRs.
TraceManifest	The genesis block of a trace. Binds a traceId to its specific Constitution, Plan, and Policy hashes.
Causal Closure	The invariant that every state-mutating SystemEvent must reference the ExecutionEvent or DecisionEvent that caused it, eliminating trust in rootless state transitions.
Event-Sourced Blackboard	A memory substrate where state is a projection of BLACKBOARD_WRITTEN system events, rather than an opaque mutated object.
Signature	A cryptographic attestation of authorship, binding an identity to an artifact or event.

4. ARCHITECTURAL INVARIANTS

4.1 Factual Reduction: The reducer consumes only SystemEvents. No other event tier may directly mutate kernel state.

4.2 Causal Closure: Every SystemEvent representing a step or workflow transition must reference the executionEventId or decisionEventId that precipitated it. Rootless state transitions are constitutionally invalid.

4.3 Constitutional Supremacy: IR precedence is strictly enforced. Conflict at any level results in an immediate FATAL_CONSTITUTIONAL_VIOLATION.

`ConstitutionIR > PolicyIR > ExecIR > HermesIR`

4.4 Single Authoritative Sequencer (Model A): To guarantee strict deterministic replay, the runtime operates on a single-authoritative-event-sequencer model. eventSequence and logicalClock are sufficient. Vector clocks are excluded from the kernel; distributed execution is achieved by delegating to isolated sub-traces or external workers whose results are normalized back into the authoritative sequence.

4.5 Event-Sourced Memory: Blackboard state is derived by folding BLACKBOARD_WRITTEN events. In-place mutation outside the event stream is prohibited.

4.6 Authorship over Integrity: Hashes prove integrity; signatures prove authority. Governance proposals, approvals, and constitutional amendments require cryptographic signatures.

5. CANONICAL IR MODEL & PRECEDENCE

5.1 ConstitutionIR

Defines runtime-wide executable axioms. The ConstitutionIR is the supreme governing artifact of a trace. All validators compiled from it run synchronously prior to any policy or execution evaluation.

```
interface ConstitutionIR {
  id: string;
  version: string;
  constitutionalHash: string;
  axioms: ConstitutionalAxiom[];
}

interface ConstitutionalAxiom {
  id: string;
  description: string;
  severity: 'FATAL' | 'ERROR' | 'WARNING';
  validator: string; // URI or hash of a deterministic validation function
}
```

Example Axiom (Severity: FATAL): "No STEP_SUCCEEDED event is valid without a preceding ExecutionEvent referencing the same stepId."

5.2 HermesIR (Intent), ExecIR (Mechanics), PolicyIR (Law)

As defined in v1.3, but strictly subordinate to ConstitutionIR. These IRs govern intent expression, execution mechanics, and runtime law respectively. No construct within these IRs may supersede or conflict with a constitutional axiom. Violations are surfaced as FATAL_CONSTITUTIONAL_VIOLATION kernel halts per Invariant 4.3.

6. TRACEMANIFEST (GENESIS BLOCK)

Before execution begins, a trace must be instantiated with its constitutional and operational anchors. The TraceManifest is the first artifact written and is immutable for the lifetime of the trace.

```
interface TraceManifest {
  traceId: string;
  planId: string;
  constitutionHash: string;
  executionPlanHash: string;
  policyHash: string;
  initiatedBy: Signature;
  createdAt: number;
}
```

Replay validation begins by loading the TraceManifest to verify that subsequent events adhere to the correct constitution and plans. Any mismatch between the manifest hashes and the artifacts available at replay time constitutes a fatal integrity violation.

7. KERNEL STATE & EVENT-SOURCED BLACKBOARD

7.1 KernelState

The canonical kernel state is the complete projection of all SystemEvents consumed by the reducer since the genesis block.

```
interface KernelState {
  workflowStatus: 'INITIALIZED' | 'RUNNING' | 'BLOCKED' | 'COMPLETED' | 'FAILED';
  stepStatuses: Record<string, 'PENDING' | 'RUNNING' | 'COMPLETED' | 'FAILED'>;
  blackboard: KernelBlackboard; // Projection of BLACKBOARD_WRITTEN events
  eventSequence: number;
}
```

7.2 Blackboard as Projection

The KernelBlackboard is no longer mutated directly. It is the left-fold of BLACKBOARD_WRITTEN events. Every slot write carries provenance metadata and a version counter to enable conflict detection and auditing.

```
interface BlackboardWritePayload {
  slot: string;
  value: unknown;
  previousVersion: number;
  newVersion: number;
  provenance: {
    sourceType: 'USER' | 'TOOL' | 'MODEL' | 'SYSTEM';
    sourceId: string;
    confidence?: number;
  };
}
```

8. EVENT TAXONOMY & CAUSAL LINKING

Events are strictly tiered. Only SystemEvents are valid reducer inputs. SystemAction values are an explicit, closed set of primitives — no generic details blobs are permitted at the type level.

8.1 Base Event

All events in the Hermes Runtime extend BaseEvent. The cryptographic chain linkage (eventHash / previousEventHash) enables tamper detection across the full trace.

```
interface BaseEvent {
  eventId: string;
  traceId: string;
  eventSequence: number;
  causationId: string | null;
  correlationId: string;
  tier: EventTier;
  timestamp: number;
  logicalClock: number;
  eventHash: string;
  previousEventHash: string | null;
  signatures?: Signature[]; // Required for governance/approval events
}
```

8.2 SystemEvent (The Only Reducer Input)

State transitions are primitive and explicit. No generic details blobs at the type boundary. Each SystemAction has a strictly typed payload schema in the implementation layer.

```
type SystemAction =
  | 'WORKFLOW_INITIALIZED' | 'WORKFLOW_BLOCKED' | 'WORKFLOW_FAILED' | 'WORKFLOW_COMPLETED'
  | 'STEP_STARTED' | 'STEP_SUCCEEDED' | 'STEP_FAILED'
  | 'BLACKBOARD_WRITTEN' | 'BLACKBOARD_CLEARED'
  | 'POLICY_VIOLATION' | 'CHECKPOINT_REACHED';

interface SystemEvent extends BaseEvent {
  tier: 'SYSTEM';
  payload: {
    action: SystemAction;
    // Causal closure: The event that authorized this state mutation
    executionEventId?: string; // Required for STEP_SUCCEEDED/FAILED
    decisionEventId?: string;  // Required for STEP_STARTED
    details?: Record<string, unknown>; // Strictly typed per action in implementation
  };
}
```

8.3 ExecutionEvent

Records the factual result of a side-effect boundary. An ExecutionEvent is the ground truth of what occurred at a tool invocation. It is the mandatory antecedent of any STEP_SUCCEEDED or STEP_FAILED system event.

```
interface ExecutionEvent extends BaseEvent {
  tier: 'EXECUTION';
  payload: {
    toolId: string;
    invocationId: string;
    inputs: unknown;
    outputs: unknown;
    status: 'SUCCESS' | 'FAILURE';
  };
}
```

9. SIGNATURE MODEL

Integrity (hashes) prevents tampering. Signatures establish authority. These are orthogonal concerns: a valid hash on a fraudulent artifact is meaningless without a corresponding valid signature from an authorized identity.

```
interface Signature {
  signerId: string;
  algorithm: string;
  value: string; // Cryptographic signature over the artifact hash
  timestamp: number;
}
```

Signature requirements:

- GovernanceProposal must be signed by the proposing Council Agent.
- REQUIRE_APPROVAL outcomes must be resolved via a signed DecisionEvent from an authorized human or system.
- ConstitutionIR amendments require quorum signatures.

10. POLICY & CONSTITUTIONAL EVALUATION

10.1 Precedence Evaluation Flow

When a DecisionEvent occurs, the following evaluation sequence is strictly enforced in order:

1. Constitutional Check: Does the decision violate a ConstitutionIR axiom?
    - Yes (FATAL): Halt kernel. Emit FATAL_CONSTITUTIONAL_VIOLATION.
    - Yes (ERROR/WARNING): Emit violation event, proceed or block based on severity logic.
2. Policy Check: Does PolicyIR permit the execution?
    - DENY: Emit POLICY_VIOLATION, transition step to FAILED.
    - REQUIRE_APPROVAL: Pause until signed approval DecisionEvent arrives.
    - ALLOW / CONSTRAINED_ALLOW: Proceed to ExecutionBoundary.

> **PURITY INVARIANT**
> Both Constitution and Policy evaluation must remain pure, deterministic, network-isolated, and side-effect free. Any evaluator that performs I/O, network access, or non-deterministic computation is constitutionally invalid and must be rejected at load time.

11. EXECUTION BOUNDARY & CAUSAL CLOSURE

The execution boundary guarantees that state transitions only occur as a verified, recorded consequence of side-effects. No state may change without a provable causal antecedent in the event stream.

> **CLOSURE ENFORCEMENT RULE**
>
> If a SystemEvent with action STEP_SUCCEEDED or STEP_FAILED is fed into the reducer, and its executionEventId does not point to a valid ExecutionEvent in the trace, the reducer must reject the event and throw a FATAL_CONSTITUTIONAL_VIOLATION. There is no recovery path from a rootless state transition.

12. REPLAY CONTRACT

Replay reconstructs the runtime deterministically from the TraceManifest and the factual event stream. A replay that does not produce the recorded terminal state is evidence of either implementation divergence or event stream corruption — both of which are fatal.

12.1 Replay Steps

1. Load TraceManifest — Verify genesis hashes against available artifacts.
2. Initialize state from Checkpoint — If applicable; otherwise initialize from scratch.
3. Consume events strictly by eventSequence — No out-of-order processing permitted.
4. Filter for SystemEvent tier only — All other tiers are ignored by the reducer.
5. Validate Causal Closure — Ensure STEP_SUCCEEDED / STEP_FAILED events reference valid ExecutionEvents within the trace.
6. Feed into reducer — Apply each SystemEvent to produce the next KernelState.
7. Validate cryptographic chain linkage — Verify eventHash / previousEventHash integrity across the full sequence.
8. Assert deep equality — Compare computed terminal state against recorded terminal state. Any divergence is a fatal replay violation.

13. FINAL STATEMENT

"Hermes is a fact-derived reality engine."

Models propose. Policies judge. Executions occur. Facts are recorded. Reducers derive reality.

By enforcing causal closure, executable constitutions, explicit primitive transitions, and event-sourced memory, the system eliminates the ambiguity that allows probabilistic components to corrupt deterministic state. The runtime does not guess what happened; it cryptographically proves what occurred, strictly ordered, and derives the present from that proof.

IMPLEMENTATION SPECIFICATION

DOCUMENT 3 :: RFC-002 CANONICAL EVENT MODEL, CRYPTOGRAPHIC CHAINING, AND SERIALIZATION

HERMES PLATFORM ENGINEERING SERIES · REF: RFC-002

DOCUMENT ID	RFC-002	**EFFECTIVE DATE**	June 14, 2026
REVISION	v1.0	**STATUS**	Implementation Specification
CLASSIFICATION	Internal Engineering Ref	**RELATIONSHIP**	Implements RFC-001

> ▶ **STRATEGIC FRAMING**
> "This is the exact correct inflection point. The constitution is written; now we must define the physics. Phase 0 is the only priority. If the kernel cannot prove reality from facts, the rest of the stack is science fiction."

1. ABSTRACT

This RFC defines the exact mechanical representation of truth in the Hermes Runtime. If RFC-001 is the constitutional law, RFC-002 is the criminal code — it specifies the precise schemas, serialization rules, cryptographic algorithms, and validation procedures required to ensure that a recorded trace can be replayed to produce byte-identical state reconstruction.

2. SERIALIZATION AND CANONICALIZATION

To guarantee deterministic replay across different languages, runtimes, and timestamps, events must be canonicalized before hashing or signing.

- **2.1 Format:** All events are serialized as JSON strings.
- **2.2 Canonicalization Algorithm:** We adopt RFC 8785 (JSON Canonicalization Scheme — JCS). JCS ensures that JSON payloads with identical logical content produce identical byte representations by strictly defining whitespace, string escaping, and property ordering.
- **2.3 Numeric Constraints:** To prevent floating-point drift across runtimes:
  - All timestamps MUST be represented as 64-bit integers (Unix epoch milliseconds).
  - No floating-point numbers are permitted in canonicalized payloads. Decimals (e.g., costs, confidence scores) MUST be represented as strings (e.g., "0.95") or scaled integers.

3. CRYPTOGRAPHIC HASHING AND CHAINING

The audit chain guarantees the temporal integrity and tamper-evidence of a trace.

3.1 Hashing Algorithm: SHA-256.

3.2 Chain Construction: Every event is linked to its predecessor within the same traceId. The construction procedure is as follows:

1. Construct the event object, leaving `eventHash` and `previousEventHash` as empty strings ("").
2. Replace `previousEventHash` with the `eventHash` of the preceding event in the trace. For the first event in the trace, use the hash of the TraceManifest.
3. Canonicalize the object using JCS, excluding the still-empty `eventHash` field.
4. Compute the SHA-256 hash of the canonicalized UTF-8 string.

5. Assign the resulting hex-encoded string to `eventHash`.

Implementation Note: The eventHash field is explicitly excluded from the hash input to prevent recursive computation. The hash covers previousEventHash, ensuring the chain is unbroken.

3.3 Chain Validation Procedure

During replay, the engine must verify the chain using the following procedure:

1. Load TraceManifest. Compute its hash.
2. For the first event, verify previousEventHash equals the manifest hash.
3. For event N, verify previousEventHash equals the eventHash of event N-1.
4. Recompute the hash of event N (excluding eventHash) and verify it matches the stored eventHash.

> ⚠ **FATAL ERROR CONDITION — CHAIN VALIDATION FAILURE**
> A chain validation failure constitutes a FATAL_CONSTITUTIONAL_VIOLATION. The trace is considered corrupted or tampered with. Replay must abort immediately. No partial state reconstruction is permitted under any failure condition. The trace must be quarantined and escalated for forensic investigation.

4. EVENT IDENTITY AND CAUSALITY

4.1 Event ID: `eventId` MUST be a UUIDv7 (time-ordered). This guarantees that event IDs are globally unique, monotonically increasing within a clock drift boundary, and sort-friendly for database indexing.

4.2 Causal Linkage: The following fields encode the causal graph of every event:

FIELD	PURPOSE
causationId	The `eventId` of the specific event that directly triggered this event. For example, a SystemEvent for step completion has causationId pointing to the ExecutionEvent.
correlationId	The `eventId` of the originating user intent or root trigger. Threads the full causal ancestry back to its origin.

5. CANONICAL EVENT SCHEMAS

All event types in the Hermes Runtime are derived from a common BaseEvent structure. The following table provides a tier hierarchy overview before the full schema definitions.

TIER	DESCRIPTION	DRIVES STATE?
MODEL	Raw probabilistic output from the LLM.	No
DECISION	Agent action choice derived from model output.	No
EXECUTION	Tool side-effect result; factual record of what happened.	No
SYSTEM	State transition fact emitted by the kernel.	**YES (Sole Input)**
TELEMETRY	Metrics and operational data; observability only.	No

5.1 BaseEvent

All events inherit this structure. Every field defined here is mandatory on every emitted event unless explicitly marked optional.

```
interface BaseEvent {
  eventId: string;           // UUIDv7
  traceId: string;
  eventSequence: number;     // Strictly monotonic within trace
  causationId: string | null;
  correlationId: string;
  tier: EventTier;
  timestamp: number;         // Unix epoch ms (int64)
  logicalClock: number;      // Monotonically increasing kernel tick
  eventHash: string;         // SHA-256 hex
  previousEventHash: string; // SHA-256 hex of previous event or manifest
  signatures?: Signature[];  // Optional: Required for GOV/APPROVAL tiers
}
```

5.2 ModelEvent

Raw probabilistic output from the LLM. This tier is observational — it drives no state transitions directly.

```
interface ModelEvent extends BaseEvent {
  tier: 'MODEL';
  payload: {
    agentId: string;
    modelId: string;
    promptHash: string;      // SHA-256 of the exact prompt template + vars
    rawOutput: string;       // Unparsed LLM text
    tokenUsage: { input: number; output: number; };
  };
}
```

5.3 DecisionEvent

The runtime's choice of action based on model output. Records the provenance of every decision including which model event produced the candidates and which was selected.

```
interface DecisionEvent extends BaseEvent {
  tier: 'DECISION';
  payload: {
    stepId: string;
    agentId: string;
    proposedAction: string;
    targetToolId: string | null;
    parameters: Record<string, unknown>;
    provenance: {
      modelEventId: string;  // Links to the ModelEvent that generated candidates
      chosenCandidateIndex: number;
      selectionRationale: string | null;
    };
  };
}
```

5.4 ExecutionEvent

The factual result of a side-effect. This is the immutable record of what a tool did. Full inputs and outputs are content-addressed externally.

```
interface ExecutionEvent extends BaseEvent {
  tier: 'EXECUTION';
  payload: {
    stepId: string;
    toolId: string;
    invocationId: string;    // Idempotency key for the tool
    inputsHash: string;      // SHA-256 of canonicalized inputs
    outputsHash: string;     // SHA-256 of canonicalized outputs
    outputsSummary: string;  // Truncated/scrubbed output for audit
    status: 'SUCCESS' | 'FAILURE' | 'TIMEOUT';
    durationMs: number;
  };
}
```

5.5 SystemEvent — The Reducer Input

The only tier that drives KernelState mutations. All state transitions in the Hermes kernel are caused exclusively by SystemEvent records passing through the reducer.

```
type SystemAction =
  | 'WORKFLOW_INITIALIZED' | 'WORKFLOW_COMPLETED' | 'WORKFLOW_FAILED' | 'WORKFLOW_BLOCKED'
  | 'STEP_STARTED' | 'STEP_SUCCEEDED' | 'STEP_FAILED'
  | 'BLACKBOARD_WRITTEN' | 'POLICY_VIOLATION' | 'CHECKPOINT_REACHED';

interface SystemEvent extends BaseEvent {
  tier: 'SYSTEM';
  payload: {
    action: SystemAction;
    stepId?: string;
    executionEventId?: string; // Causal closure mandates: SUCCEEDED/FAILED must reference
ExecutionEvent
    blackboardMutation?: {
      slot: string;
      valueHash: string;
      previousVersion: number;
      newVersion: number;
      provenance: {
        sourceType: 'USER' | 'TOOL' | 'MODEL' | 'SYSTEM';
        sourceId: string;
        confidence: number | null;
      };
    };
  };
}
```

5.6 TelemetryEvent

Metrics and operational data. Observability-only.

```
interface TelemetryEvent extends BaseEvent {
  tier: 'TELEMETRY';
  payload: {
    metric: string;
    value: number;
    unit: string;
    tags: Record<string, string>;
  };
}
```

6. EVENT SIGNING

Hashes prove integrity; signatures prove authority. The two mechanisms are complementary and serve distinct security properties.

- **6.1 Algorithm:** Ed25519 — deterministic, fast, and widely supported.
- **6.2 Signing Procedure:** The signer computes the eventHash, signs the UTF-8 hex string of the eventHash using their private key, and appends the signature object to the signatures array.

```
interface Signature {
  signerId: string;      // SPIFFE ID or Agent ID
  algorithm: 'Ed25519';
  value: string;         // Hex-encoded signature
  timestamp: number;     // Unix epoch ms
}
```

6.3 Required Signatures: WORKFLOW_INITIALIZED MUST be signed by the runtime initiator. STEP_SUCCEEDED arising from a REQUIRE_APPROVAL policy MUST include a signature from the human approver authorizing the ExecutionEvent.

7. CHECKPOINT LINKAGE

Checkpoints bound the cost of replay. They allow the replay engine to fast-forward to a known-good state.

7.1 Checkpoint Creation

A SystemEvent with action: CHECKPOINT_REACHED is emitted.

```
interface CheckpointPayload {
  action: 'CHECKPOINT_REACHED';
  checkpointId: string;
  eventSequence: number;     // The sequence number of this event
  kernelStateHash: string;   // SHA-256 of the canonicalized KernelState at this tick
  blackboardHash: string;    // SHA-256 of the canonicalized Blackboard at this tick
}
```

8. REPLAY VALIDATION PROCEDURE — THE PROOF

> **◆ PHASE 0 CRITICAL DELIVERABLE**
>
> This section defines the central deliverable of Phase 0. The `validateTrace` function below is the proof that the Hermes physics engine is real. Until this function runs against a live trace and returns true, all other platform work is speculative.

To prove a trace is valid, the replay engine must execute the following algorithm in its entirety. Each check is a constitutional axiom — no check may be skipped or downgraded to a warning.

```
function validateTrace(manifest: TraceManifest, events: BaseEvent[]): boolean {
  let prevHash = hashManifest(manifest);
  let expectedSequence = 1;
  for (const event of events) {
    // 1. Sequence Check
    if (event.eventSequence !== expectedSequence) throw new Error("Sequence gap");
    // 2. Chain Integrity
    if (event.previousEventHash !== prevHash) throw new Error("Chain broken");
    // 3. Hash Verification
    const computedHash = computeEventHash(event);
    if (computedHash !== event.eventHash) throw new Error("Tampered event");
    // 4. Causal Closure (Constitutional Axiom)
    if (event.tier === 'SYSTEM') {
      const sysEvent = event as SystemEvent;
      if (sysEvent.payload.action === 'STEP_SUCCEEDED' || sysEvent.payload.action === 'STEP_FAILED')
{
        if (!sysEvent.payload.executionEventId) {
            throw new Error("Causal closure violation: rootless state transition");
        }
        // Verify the referenced ExecutionEvent exists in the past events
        if (!events.find(e => e.eventId === sysEvent.payload.executionEventId)) {
            throw new Error("Phantom execution reference");
        }
      }
    }
    prevHash = computedHash;
    expectedSequence++;
  }
  return true;
}
```

9. IMPLEMENTATION CHECKLIST

The following files constitute the complete Phase 0 implementation surface. Every file listed below must exist and pass a test suite before Phase 0 is considered complete.

FILE	RESPONSIBILITY (PHASE 0)
types.ts	All TypeScript interfaces defined in Section 5: BaseEvent, ModelEvent, DecisionEvent, ExecutionEvent, SystemEvent, TelemetryEvent, Signature.
hashing.ts	JCS canonicalization, SHA-256 hash computation, computeEventHash() per Section 3.2, manifest hashing, and Ed25519 signing utilities per Section 6.
event-store.ts	Event persistence layer. Append-only write path, indexed reads by traceId and eventSequence, and hash-keyed object storage delegation for full I/O payloads.
reducer.ts	Pure function: (KernelState, SystemEvent) => KernelState. Handles all SystemAction variants. Must be side-effect free and deterministic.
replay.ts	Implements validateTrace() per Section 8. Sequence check, chain integrity, hash verification, causal closure enforcement. Must throw on any violation.
checkpoint.ts	Checkpoint creation (Section 7.1): snapshot KernelState, compute hashes, emit CHECKPOINT_REACHED SystemEvent. Checkpoint-based replay fast-forward (Section 7.2).

IMPLEMENTATION SPECIFICATION

DOCUMENT 4/5 :: RFC-003 & 004 ALGEBRAIC ORDERING & DISTRIBUTED CLOSURE

HERMES PLATFORM ENGINEERING SERIES · REF: RFC-003/004

DOCUMENT ID	RFC-003/004	**EFFECTIVE DATE**	June 14, 2026
REVISION	v1.0	**STATUS**	Canonical Mathematical Foundation
CLASSIFICATION	Internal Engineering Ref	**SCOPE**	Causal Topology & Linearization

> ► **STRATEGIC FRAMING**
> This document bridges the gap from structured architecture to runtime physics. It formally defines the algebraic objects, the causal partial order, the linearization function, and the exact homomorphism requirements of the distributed reducer.

1. FIRST PRINCIPLES: THE EXECUTION MODEL

The Hermes Runtime operates on three distinct mathematical layers. Failure to separate these layers results in crystallization ambiguity and hidden nondeterminism.

1. **The Causal Partial Order (`P`):** The true topological dependency graph of events. Defined by the relation `<` (happened-before).
2. **The Total Linear Extension (`L`):** A strictly sequential, cryptographically chained ordering of events. This is a valid topological sort of `P`.
3. **The State Projection (`Σ`):** The deterministic fold of the Reducer function over the Total Linear Extension.

The fundamental theorem of the Hermes substrate is:

> Any valid linear extension L of a causal graph P fed into a pure Reducer f will yield a verifiable state Σ.

2. EVENT TOPOLOGY AND THE CAUSAL RELATION (<)

To support parallel steps, fork-join workflows, and speculative execution, the single `causationId` pointer is elevated to a `parentIds` set.

2.1 The Causal Relation

For any event `e`, let `P(e)` be its `parentIds`. We define the "happened-before" relation `<` as the transitive closure of the parent graph:

$$e_1 < e_2 \Leftrightarrow e_1 \in P(e_2) \vee \exists\ e_3 : (e_1 < e_3 \wedge e_3 < e_2)$$

2.2 Invariant: Causal Acyclicity

The set of all events `E` in a trace must form a Directed Acyclic Graph (DAG). If a proposed event creates a cycle, it is mathematically invalid and must be rejected.

$$\nexists\ e \in E \text{ such that } e < e$$

2.3 Invariant: Parent Existence

All parents must exist in the committed prefix before an event is crystallized.

$$\forall\ p \in P(e),\ p \text{ must exist in } E$$

3. THE LINEARIZATION FUNCTION (L)

Because distributed systems cannot always agree on a single global timeline instantly, nodes may observe partial, concurrent histories. The runtime requires a pure, deterministic linearization function L.

3.1 Definition

L takes a set of unsequenced proposals (a partial order) and returns a strictly ordered array of events (a total order).

3.2 The Tie-Breaking Rule (Crystallization Determinism)

When L encounters events that are concurrent ($e_1 \not< e_2$ and $e_2 \not< e_1$), it must resolve the order deterministically without relying on wall-clock timestamps or network arrival order.

$$\text{if } e_1 \not< e_2 \text{ and } e_2 \not< e_1, \text{ sort by } \text{SHA-256}(\text{JCS}(e_1)) < \text{SHA-256}(\text{JCS}(e_2))$$

This ensures that `crystallize()` is mathematically pure. Any two nodes resolving the exact same set of concurrent proposals will yield the identical total order.

4. THE BIFURCATION: AUTHORITY VS. ALGEBRA

The kernel accepts events from an external ordering source. This source is treated as an untrusted oracle.

4.1 Object A: The Ordering Authority (Total Order Oracle)

- **Context:** Phase 0 / Single-Writer / Raft-based consensus.
- **Contract:** The oracle provides exactly one event at a time. It guarantees that no concurrent events exist.
- **Kernel Validation:** 1) `event.prevHash === tip.hash`, 2) `event.sequence === tip.sequence + 1`, 3) `parentIds` are strictly in the past.

4.2 Object B: The Ordering Algebra (DAG Resolver)

- **Context:** Phase 1+ / Multi-Writer / DAG-based consensus.
- **Contract:** The algebra accepts concurrent proposals, constructs the DAG, and applies L to yield a batch of sequenced events.
- **Kernel Validation (Strict):** The Kernel treats the batch as an untrusted block. It verifies chain links, continuity, and crucially, **verifies the Linearization**. The kernel checks that the batch provided strictly adheres to the topological sort of $<$ and the deterministic tie-breaking rule.

5. CANONICAL INVARIANTS TABLE

5.1 Causal Correctness

- **[C-01] DAG Integrity:** Event parent graphs must be acyclic.
- **[C-02] Causal Closure:** Every state-mutating `SystemEvent` must reference an `ExecutionEvent` in its `parentIds`.
- **[C-03] Execution Idempotency:** For any `(toolId, invocationId)` pair, there exists at most one `SUCCESS` `ExecutionEvent` in the entire causal history.

5.2 Ordering Correctness

- **[O-01] Total Order Integrity:** `event.prevHash === H(event_n - 1)`
- **[O-02] Monotonic Sequence:** `event.sequence === event_n - 1.sequence + 1`
- **[O-03] Valid Extension:** The sequenced batch is a valid topological sort of the underlying DAG (P).

5.3 Execution Correctness

- **[E-01] Hash Binding:** `event.hash === H(JCS(event \ {hash}))`
- **[E-02] Attestation Binding:** Signatures are valid and correctly bound to the `event.hash`.
- **[E-03] Zero-Drift:** No floats, `NaN`, or `Infinity` in canonical state.

6. DISTRIBUTED CAUSAL CLOSURE & CONVERGENCE SPECIFICATION

6.1 The Visibility Function (V)

Let E be the global set of valid events. Let N be a node in the network. The visibility function $V_N(t) \subseteq E$ returns the event set observable by node N at logical time t.

The Convergence Condition (Global Truth): The system achieves global determinism if and only if all nodes converge on the identical linear extension L.

$$\lim_{t \to \infty} V_N(t) = E \quad \forall N$$

6.2 The Crystallization Closure Operator (CC)

Because $V_N(t)$ is often incomplete, a node cannot simply sort $V_N(t)$. It must only crystallize events whose causal past is fully known. We define the Crystallization Closure:

$$CC(V_N(t)) = \{ e \in V_N(t) \mid P^*(e) \subseteq V_N(t) \}$$

Where $P^*(e)$ is the recursive transitive closure of all parents of e.

Invariant [C-04]: The Ordering Algebra *must not* return an event for crystallization unless it belongs to $CC(V_N(t))$.

7. O(N) TOPOLOGICAL VERIFICATION (THE REACHABILITY ORACLE)

To prevent exponential recomputation during kernel verification, the kernel does not validate the entire DAG. It validates the linear extension against a strictly bounded **Causal Frontier**.

7.1 The Causal Frontier (F)

$$F = \{ e \in E_{crystallized} \mid \nexists e' \in E_{crystallized} \text{ where } e \in P(e') \}$$

7.2 The O(n) Verification Algorithm

When the Kernel receives a candidate batch of ordered events $B = [e_1, e_2, \ldots, e_k]$ from the Algebra, it verifies topological validity in exactly $O(|B|)$ time.

1. Initialize `processed_set` = F.
2. For each event e_i in batch B:
    - Check $P(e_i) \subseteq$ `processed_set`.
    - If yes, add e_i to `processed_set`.
    - If no, throw `CAUSAL_VIOLATION`.

8. REDUCER HOMOMORPHISM OVER LINEARIZATIONS

For the system to yield identical state Σ regardless of which valid topological sort $L \in L(P)$ is chosen, the Reducer f must be a **functor preserving equivalence classes of linearizations**.

$$\forall\ L_1, L_2 \in L(P),\ f(L_1) = f(L_2)$$

8.1 Enforcement Constraint: Disjoint State Algebra

If two events e_1 and e_2 are concurrent ($e_1 \not\prec e_2 \wedge e_2 \not\prec e_1$), their execution is independent. For f to remain homomorphic over them, **they must not touch the same keys in the Blackboard state space**. Let $W(e)$ be the set of state keys mutated by event e.

$$\forall\ e_1, e_2 \text{ where } e_1 \not\prec e_2 \wedge e_2 \not\prec e_1,\ W(e_1) \cap W(e_2) = \varnothing$$

Kernel Enforcement: If the kernel detects a batch where two concurrent events attempt to mutate the same Blackboard slot, it throws a `STATE_HOMOMORPHISM_VIOLATION`. Conflicting operations *must* be explicitly sequenced via `parentIds` in the ExecIR.

9. IMPLEMENTATION: THE FINAL TOPOLOGICAL OBJECTS

9.1 Updated Types (topology.ts)

```
export type EventId = string;

export interface CausalEvent {
  id: EventId;
  traceId: string;
  parentIds: EventId[]; // Replaces single causationId
  action: Action;
  payload: unknown;
  hash: string;         // Computed over JCS({id, traceId, parentIds, action, payload})
}

export interface KernelMetaState {
  lastAppliedSequence: number;
  lastAppliedEventHash: string;
  // The Reachability Oracle (The Frontier)
  causalFrontier: Set<EventId>;
}
```

9.2 The Topological Validator (validator.ts)

The `O(n)` batch verification logic that enforces the Closure and Homomorphic constraints.

```
export class TopologicalValidator {
  /**
   * Validates a crystallized batch from the Ordering Algebra.
   * Enforces [O-03] Valid Extension in O(n) time using the Frontier.
   */
  public static validateBatch(
    batch: CausalEvent[],
    currentFrontier: Set<EventId>,
    blackboardMutations: Map<string, EventId>
  ): void {

    const processedInBatch = new Set<EventId>();
    const concurrentMutations = new Map<string, EventId>();

    for (const event of batch) {
      // 1. Verify Causal Closure (Parents must be in Frontier OR processed earlier in batch)
      for (const parentId of event.parentIds) {
        if (!currentFrontier.has(parentId) && !processedInBatch.has(parentId)) {
          throw new Error(`CAUSAL_VIOLATION: Parent ${parentId} missing for event ${event.id}`);
        }
      }

      // 2. Verify Homomorphic State Safety
      // If this event mutates a state key, ensure no un-sequenced (concurrent) peer has mutated it.
      const mutatedKeys = TopologicalValidator.getMutatedKeys(event);
      for (const key of mutatedKeys) {
        if (concurrentMutations.has(key)) {
          throw new Error(`HOMOMORPHISM_VIOLATION: Concurrent write to key ${key}`);
        }
        concurrentMutations.set(key, event.id);
      }

      processedInBatch.add(event.id);
    }
  }

  private static getMutatedKeys(event: CausalEvent): string[] {
    // Extracts keys from payload (e.g., BLACKBOARD_WRITTEN)
    // Implementation omitted for brevity
    return [];
  }
}
```

9.3 The Ordering Algebra Contract (algebra.ts)

```
export interface OrderingAlgebra {
  /**
   * Injects a proposed intent into the Causal Cone.
   * The Algebra MUST buffer this until its parents are satisfied.
   */
  propose(event: CausalEvent): void;

  /**
   * The Crystallization Operator (CC).
   * MUST only return events whose full transitive closure is known.
   * MUST sort concurrent events deterministically by SHA256(event.hash).
   */
  crystallize(): CausalEvent[];
}
```